Mikhail Razakov

C++ / Systems Engineer — LLVM · Compiler Tooling

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop C++/LLVM tools for program analysis and conservative IR transformations. I worked on experimental LLVM LICM during my internship at MCST and now work on symbolic computation in SharpChecker (ISP RAS). My independent projects cover memory-dependence analysis (GNU C17), LLVM Global IV (C++23) and a Rust x86-64 backend. I use regression and differential tests to evaluate correctness.

TECHNICAL SKILLS

C++ and tooling: C++23, GNU C17 (POSIX), LLVM 22/IR, CMake, GCC/Clang, Git, Linux, Python

Algorithms and compilers: CFG/SSA, dominance, data-flow, interprocedural effects, memory dependence; graph algorithms, Rust/x86-64 SysV ABI

Verification: Regression and differential testing, reference oracles, ASan/UBSan, counterexamples and conservative transformation-legality checks

EXPERIENCE

ISP RAS · SharpChecker · October 2026 — present

- Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship · July — August 2026

- Worked on an experimental LLVM LICM transformation, checking loop structure, memory accesses and transformation legality; compared before/after behavior using Python.
- Implemented RPO, Dijkstra, Dinic and Tarjan SCC in C++23; the local run from a Git snapshot passed 14/14 fixture tests.

ENGINEERING PROJECTS

LLVM Interprocedural Global IV Optimization

An LLVM 22 pass promotes selected integer-global accesses within loops. Effect analysis and a separate PromotionLegality gate precede IR rewriting. Passed 29/29 positive and conservative-rejection regression scenarios on pinned LLVM 22.1.8.

PS-form Memory Dependence Analyzer

A GNU C17 analyzer for overlapping memory accesses across distinct loop iterations, combining exact counting for supported affine cases with modular/GCD checks and bounded fallback. Unresolved cases return conservative maybe. Built as an MCST technical-selection task, not a production dependence-analysis system.

x86-64 Compiler Backend & Register Allocation

An experimental Rust backend for restricted IR with SSA/CFG, liveness, register allocation, spills, phi lowering and SysV x86-64 code generation. An independent verifier checks register/stack assignments; native results are compared against a reference IR interpreter.

EDUCATION

HSE University — Software Engineering

RECOGNITION

2026 · MCST LLVM/C++ selection: According to organizer correspondence: the only 5.0/5.0 score and only 104/104 official-test submission among 49 entries.

2025–2026 · MEPhI Junior: First-degree diplomas in the Information Technologies section in both 2025 and 2026.

2026 · LangDev — UniversalToolchain: Presented on 9 October 2026; the organizer published a talk recording. — Talk recording